

lists of items. Also we will need to do this on pages that list published articles categories etc. The Django template language has a for loop block that can iterate over a lists of items. So to print a list of all the articles tags, you could do the following:

```
• {% for tag in article.tags.all %}
•     {{ tag.name }}
• {% empty %}
•     No tags
• {% endfor %}
```

Note that here we have also supplied a message to display if there are no items in the list, this is optional.

Another thing we may want to do is to keep track of the loop counter, and possibly display it or use it in some other way. This counter can be accessed via ‘forloop.counter’ if you want a counter starting from 1 or ‘forloop.counter0’ if you want a counter starting at 0. In either case you can print this counter by placing it inside double curly braces as usual.

URL

Another incredibly important thing we’d want to do is to link between different kinds of data in our database. We’d want all articles in the article list to be linked to articles, and all tags on an article page to go to that tag’s page etc. For this we need to be able to get the URL of these objects to include within a link tag in our code.

If you’ll recall we mentioned that you should provide a name for all your views, and this is the reason why, if you know the name of a URL pattern, you can generate a URL for it given all the parameters. For instance to generate the URL to a particular article we can do the following:

```
• {% url 'articles-detail' article.slug %}
```

Here we are assuming that the name of the URL pattern for the article details page is ‘articles-detail’. You could use this to generate a link with the following code:

```
• <a href="{% url 'articles-detail' article.slug %}">
•     {{ article.title }}
• </a>
```

Filters

Filters are another important part of Django templates. They allow us to process variables before being displayed or used in our templates. For example,